Week 2 Assignment

Context:

As an Automation Architect, generate me a Playwright framework structure for Tableau to Power BI report migration project.

Instruction:

ASK me 5 questions one by one for framework creation -Ask me the next question after me providing the answer -Framework should be Enterprise following the industry standard Step by step implementation

Question 1: Primary Automation Scope

What exactly do you want to automate in this migration validation?

Choose all that apply (or describe your own):

A. UI validation (Power BI reports vs Tableau reports – visuals, filters, pagination)
B. Data validation (backend DB vs Power BI dataset)
C. API validation (Power BI REST APIs / dataset refresh APIs)
D. End-to-End validation (source → transformation → report)
E. Visual regression (chart/image comparison)
F. Export validation (PDF/Excel comparison between Tableau & Power BI)

Question 2: Technology Stack & Execution Environment

To design an enterprise-grade framework, I need to lock the execution stack:

What is your preferred stack for Playwright?

Choose or specify:

A. Language:

- Python (pytest)
- TypeScript (Playwright Test runner)

B. Execution:

- Local execution only
- CI/CD (Azure DevOps / Jenkins / GitHub Actions)

C. Data Sources for Validation:

- Azure Synapse
- Azure SQL DB
- ADLS / Data Lake
- Databricks

D. Authentication Type (very important for Power BI):

- Azure AD (OAuth / SSO)
- Service Principal
- Username/Password

Question 3: Test Design & Execution Strategy

How do you want to organize and trigger your tests?

Choose your preferred approach:

A. Test Segmentation

- Separate suites: UI, Data, E2E
- OR unified flow-based tests (single test does UI + Data validation)

B. Execution Trigger

- On every build (CI pipeline)
- Scheduled (nightly regression)
- On-demand (manual trigger)

C. Data Handling Strategy

- Static test data (JSON/CSV files)
- Dynamic data (DB-driven / queries from Synapse)
- Hybrid (both)

D. Parallel Execution

- Yes (high priority for enterprise)
- No

Question 4: Reporting, Logging & Observability

For an enterprise framework, reporting and diagnostics are critical—especially for migration validation.

What are your preferences?

A. Reporting Tool

- Allure Reports
- Playwright HTML Reports
- Custom reporting (DB / dashboard)

B. Logging Strategy

- Basic logs (console/file)
- Structured logging (log levels, JSON logs, correlation IDs)

C. Failure Diagnostics
Choose what you need:

- Screenshots on failure
- Video recording
- Trace viewer (Playwright tracing)
- Network logs

D. Integration

- Azure DevOps Test Plans integration (Yes/No)
- Results storage (Blob / DB / none)

Question 5 (Final): Framework Design Patterns & Advanced Capabilities

This determines how scalable and maintainable your framework will be.

A. Design Pattern for UI Automation

- Page Object Model (POM)
- Playwright Fixtures-based architecture (recommended for scalability)
- Hybrid (POM + Fixtures)

B. Data Validation Approach

- SQL vs SQL comparison (source vs target)

- DB vs UI validation (query → compare with UI values)
- Aggregation-level validation (counts, sums, KPIs)
- All of the above

C. Reusability & Config Management

- Environment-based configs (dev/qa/prod via config files) = Yes/No
- Secrets management (Azure Key Vault) = Yes/No

D. Advanced Features
Pick what you want:

- Retry mechanism for flaky tests
- Tag-based execution (smoke/regression)
- Test data versioning
- Schema validation (for data migration)

- # 🧩 Enterprise Playwright Framework Architecture
- ## 1. High-Level Architecture (Layered Design)

```
┌──────────────────────────────┐
│      Test Layer              │
│ (UI / Data / E2E Suites)     │
└──────────────┬───────────────┘
               │
               ▼
┌──────────────────────────────┐
│    Business Layer            │
│ (Validation / Assertions)    │
└──────────────┬───────────────┘
               │
```

```
┌──────────────────────────▼──────────────────────────┐
│        Core Framework Layer                          │
│ Fixtures | Base Classes | Utilities | Logging        │
└──────────────────────────┬──────────────────────────┘
                           │
┌──────────────────────────▼──────────────────────────┐
│      Integration Layer (Azure + External)            │
│ Synapse | ADLS | Power BI API | Key Vault | ADO      │
└──────────────────────────────────────────────────────┘
```

2. Folder Structure (Industry Standard)

```
playwright_framework/
│
├── tests/
│   ├── ui/
│   │   ├── test_pbi_reports.py
│   │
│   ├── data/
│   │   ├── test_data_validation.py
│   │
│   ├── e2e/
│   │   ├── test_migration_e2e.py
│
├── pages/                  # POM Layer
│   ├── base_page.py
│   ├── powerbi_report_page.py
```

```
│   ├── filters_component.py
│
├── fixtures/                # Playwright Fixtures
│   ├── browser_fixture.py
│   ├── auth_fixture.py
│   ├── db_fixture.py
│
├── business/                # Business Logic Layer
│   ├── report_validator.py
│   ├── data_comparator.py
│   ├── kpi_validator.py
│
├── integrations/
│   ├── azure/
│   │   ├── synapse_client.py
│   │   ├── adls_client.py
│   │   ├── keyvault_client.py
│   │
│   ├── powerbi/
│   │   ├── powerbi_api_client.py
│
├── utils/
│   ├── logger.py
│   ├── config_loader.py
│   ├── retry.py
```

```
│   ├── json_utils.py
│
├── config/
│   ├── config_dev.yaml
│   ├── config_qa.yaml
│   ├── config_prod.yaml
│
├── test_data/
│   ├── sql_queries/
│   │   ├── sales_validation.sql
│   │
│   ├── schema/
│   │   ├── expected_schema.json
│
├── reports/
│   ├── allure/
│
├── conftest.py
├── pytest.ini
├── requirements.txt
├── azure-pipelines.yml
```

⚙️ 3. Key Design Decisions (Based on Your Inputs)

✅ Hybrid Architecture

- **POM** → UI abstraction
- **Fixtures** → lifecycle, dependency injection

- **Business Layer** → reusable validation logic

✅ **Test Segregation**

- tests/ui → visual/report validation
- tests/data → DB comparisons
- tests/e2e → full pipeline validation

✅ **Dynamic Data Validation (Synapse)**

- Queries stored in /test_data/sql_queries
- Executed via synapse_client.py
- Results compared using data_comparator.py

🔐 **4. Authentication Strategy (Azure AD SSO)**

Handled via **Playwright storage state + token reuse**

Approach:

1. One-time login → save session
2. Reuse session across tests

```
# auth_fixture.py
@pytest.fixture(scope="session")
def auth_context(browser):
    context = browser.new_context(storage_state="auth.json")
    return context
```

5. Sample Test Flow (E2E)

```
def test_sales_report_validation(page, db_connection):
    # Step 1: UI Data from Power BI
    report_page = PowerBIReportPage(page)
```

```
    ui_data = report_page.get_sales_data()

    # Step 2: Backend Data from Synapse
    db_data = db_connection.execute_query("sales_validation.sql")

    # Step 3: Validation
    DataComparator.compare(ui_data, db_data)
```

6. Business Layer (Critical for Reusability)

```
class DataComparator:

    @staticmethod
    def compare(ui_data, db_data):
        assert ui_data == db_data, "Mismatch between UI and DB"
```

7. Reporting & Observability

✅ **Allure Integration**

```
pytest --alluredir=reports/allure
allure serve reports/allure
```

✅ **Structured Logging**

```
logger.info("Validation started", extra={"correlation_id": "123"})
```

✅ **Failure Evidence**

Screenshot (on failure)

Trace enabled

Logs attached to Allure

🔁 **8. Retry + Tag Execution**

pytest.ini

```
[pytest]
addopts = --reruns 2 --reruns-delay 2
markers =
  smoke
  regression
  data
  ui
```

🔄 9. Schema Validation (Migration Critical)

```
def validate_schema(actual, expected):
  assert actual.keys() == expected.keys()
```

☁️ 10. Azure Integrations

Synapse Connection

```
class SynapseClient:
  def execute_query(self, query):
    # pyodbc / sqlalchemy logic
    Pass
```

Key Vault

```
class KeyVaultClient:
  def get_secret(self, key):
    return secret
```

ADLS (if needed for file validation)

🚀 11. Azure DevOps Pipeline (CI/CD)

```
trigger:
 - main
```

```yaml
pool:
  vmImage: 'ubuntu-latest'

steps:
- task: UsePythonVersion@0
  inputs:
    versionSpec: '3.10'

- script: |
    pip install -r requirements.txt
    playwright install
  displayName: Install Dependencies

- script: |
    pytest -m "smoke" --alluredir=reports/allure
  displayName: Run Tests

- task: PublishBuildArtifacts@1
  inputs:
    pathToPublish: reports
```

🧱 12. Step-by-Step Implementation Plan

Phase 1: Setup

1. Initialize repo
2. Install Playwright + Pytest
3. Setup folder structure

Phase 2: Core Framework

4. Build fixtures (browser, auth, DB)
5. Implement config loader (env-based)
6. Setup logging (JSON structured logs)

Phase 3: UI Layer

7. Create base page + POM classes
8. Implement reusable components (filters, charts)

Phase 4: Data Layer

9. Build Synapse client
10. Store SQL queries
11. Create comparator utilities

Phase 5: Business Layer

12. KPI validation logic
13. Aggregation checks (sum/count)

Phase 6: Advanced Features

14. Retry mechanism

15. Tag execution
16. Schema validation

Phase 7: CI/CD

17. Integrate with Azure DevOps
18. Add Allure reporting
19. Store reports in Blob

🧩 13. Key Enterprise Best Practices

- Strict separation of concerns
- No assertions inside POM
- Centralized validation layer
- Config-driven execution
- Reusable SQL-based validation
- Correlation ID tracking for debugging
- Parallel execution via pytest-xdist

⚠️ Common Pitfalls (Avoid These)

- Mixing UI + DB logic in test files
- Hardcoding credentials (use Key Vault)
- Not handling SSO session reuse
- No retry → flaky pipeline
- Ignoring aggregation validation (critical for BI)

✅ Outcome

This framework will allow you to:

✓ **Validate Power BI vs Tableau migration accurately**
✓ **Achieve high reusability and scalability**
✓ **Run parallel CI pipelines in Azure DevOps**
✓ **Ensure traceability + observability (Allure + logs)**
✓ **Support future expansion (Databricks, APIs, etc.)**